# TP 2 : Tests d'Intégration & API

*TestClient · SQLite · Fixtures · Faker · JUnit XML*

**Objectif du TP**

À l'issue de ce TP, vous saurez écrire des tests d'intégration avec TestClient FastAPI, configurer une BDD SQLite isolée via les fixtures conftest.py, gérer les données de test avec des fixtures et faker, valider les schémas de réponse JSON, et générer des rapports JUnit XML pour Jenkins.

| | |
|---|---|
| **Stack** | pytest · httpx · TestClient FastAPI · SQLite :memory: · faker · pytest-mock |
| **Livrable** | Suite tests/integration/ + conftest.py complet + rapport JUnit XML |
| **Prérequis** | TP0 (ShopFlow fonctionnel) + TP1 (fixtures SQLite maîtrisées) |

## Tests Unitaires vs Tests d'Intégration

| **TP1 : Tests Unitaires** | **TP2 : Tests d'Intégration** |
|---|---|
| Teste une fonction isolée | Teste un endpoint HTTP complet |
| Mocks pour toutes les dépendances | BDD SQLite réelle (pas de mock) |
| Pas de FastAPI ni de HTTP | TestClient simule des requêtes HTTP |
| assert calcul_prix_ttc(100) == 120 | assert response.status_code == 201 |
| Très rapide (< 1ms par test) | Plus lent (10-50ms par test) |
| Couvre la logique métier | Couvre routes + services + BDD |

**Pourquoi les deux types de tests sont nécessaires ?**

- Les tests unitaires valident la logique métier en isolation : calcul de prix, gestion du stock, coupons.
- Les tests d'intégration valident que les couches fonctionnent ensemble : la route appelle bien le service, le service lit bien la BDD, la réponse est bien sérialisée par Pydantic.
- Un bug peut être invisible en test unitaire (les mocks masquent les problèmes d'intégration) mais détecté en test d'intégration.
- Exemple : calcul_prix_ttc() est correcte (TP1), mais la route /orders/ n'appelle pas calcul_prix_ttc() (bug détecté uniquement en TP2).

# Partie 1 : Configuration TestClient & Fixtures

## 1.1 TestClient FastAPI

FastAPI fournit un **TestClient** basé sur httpx qui simule des requêtes HTTP en mémoire, sans démarrer un vrai serveur. L'avantage crucial : on peut remplacer la dépendance `get_db` par une session de test via `dependency_overrides`.

Principe du TestClient

```
# Principe du TestClient avec override de dépendance
from fastapi.testclient import TestClient
from app.main import app
from app.database import get_db

# 1. Créer une BDD de test isolée
engine_test = create_engine('sqlite:///:memory:', ...)

# 2. Override : remplace get_db par notre session de test
def override_get_db():
    session = SessionTest()
    try:
        yield session
    finally:
        session.close()

app.dependency_overrides[get_db] = override_get_db
# ↑ FastAPI injectera override_get_db() dans toutes les routes

# 3. Créer le client
client = TestClient(app)

# 4. À la fin, nettoyer
app.dependency_overrides.clear()
```

## 1.2 Compléter tests/conftest.py

Le fichier `conftest.py` doit être enrichi avec les fixtures pour les tests d'intégration. Ajoutez les fixtures `client`, `api_product` et `api_coupon` au fichier existant :

tests/conftest.py (nouvelles fixtures)

```
# tests/conftest.py — ajouter ces fixtures aux fixtures TP1 existantes
from fastapi.testclient import TestClient
from app.main import app
from app.database import get_db


# TESTCLIENT FASTAPI
@pytest.fixture(scope='module')
def client(db_engine):
    """TestClient FastAPI avec BDD SQLite isolée par module de test."""
    SessionTest = sessionmaker(bind=db_engine)

    def override_get_db():
        session = SessionTest()
        try:
            yield session
        finally:
            session.close()

    app.dependency_overrides[get_db] = override_get_db
    with TestClient(app) as c:
        yield c
    app.dependency_overrides.clear()


# FIXTURES API
@pytest.fixture
```

```python
def api_product(client):
    """Crée un produit via POST /products/ et le retourne."""
    response = client.post('/products/', json={
        'name': 'Clavier Mécanique',
        'price': 89.99,
        'stock': 25,
        'category': 'peripheriques',
    })
    assert response.status_code == 201
    yield response.json()
    # Cleanup : désactiver le produit après le test
    client.delete(f'/products/{response.json()["id"]}')

@pytest.fixture
def api_coupon(client):
    """Crée un coupon TEST10 (-10%) via POST /coupons/."""
    response = client.post('/coupons/', json={
        'code': 'TEST10', 'reduction': 10.0, 'actif': True
    })
    assert response.status_code == 201
    yield response.json()
```

## 1.3 Créer tests/integration/__init__.py

```python
# tests/integration/__init__.py — fichier vide obligatoire
# (déjà créé dans TP0 — vérifier qu'il existe)
```

**Q1 : Configuration TestClient**

1.1. Ajoutez les fixtures client, api_product et api_coupon dans conftest.py. Lancez pytest --collect-only tests/integration/ et commentez la sortie.

1.2. Dans la fixture client, pourquoi utilise-t-on scope='module' et non scope='function' ? Quel impact cela a-t-il sur l'isolation entre tests ?

1.3. Dans la fixture api_product, pourquoi fait-on un assert response.status_code == 201 à l'intérieur de la fixture et non dans le test ?

# Partie 2 : Tests des Endpoints /products

Créez le fichier `tests/integration/test_products_api.py`. Cette suite couvre les 5 opérations CRUD sur le catalogue produits.

## 2.1 Tests de liste et de lecture

tests/integration/test_products_api.py — lecture

```python
# tests/integration/test_products_api.py
import pytest

@pytest.mark.integration
class TestListProducts:

    def test_liste_vide_au_demarrage(self, client):
        """Sans données, GET /products/ retourne une liste vide."""
        response = client.get('/products/')
        assert response.status_code == 200
        assert isinstance(response.json(), list)

    def test_produit_cree_apparait_dans_liste(self, client, api_product):
        response = client.get('/products/')
        assert response.status_code == 200
        ids = [p['id'] for p in response.json()]
        assert api_product['id'] in ids

    def test_filtre_par_categorie(self, client):
        # Créer un produit avec catégorie spécifique
        client.post('/products/', json={
            'name': 'GPU RTX', 'price': 799.0, 'stock': 3, 'category': 'gpu'
        })
        response = client.get('/products/?category=gpu')
        assert response.status_code == 200
        for p in response.json():
            assert p['category'] == 'gpu'

    def test_pagination_limit(self, client):
        for i in range(5):
            client.post('/products/', json={'name': f'Prod{i}', 'price': 10.0, 'stock': 1})
        response = client.get('/products/?limit=2')
        assert response.status_code == 200
        assert len(response.json()) <= 2

    def test_pagination_skip(self, client):
        """skip=1000 → liste vide ou résultats après le 1000e."""
        response = client.get('/products/?skip=1000&limit=10')
        assert response.status_code == 200
        assert isinstance(response.json(), list)

@pytest.mark.integration
class TestGetProduct:

    def test_get_produit_existant(self, client, api_product):
        pid = api_product['id']
        response = client.get(f'/products/{pid}')
        assert response.status_code == 200
        data = response.json()
        assert data['id'] == pid
        assert data['name'] == api_product['name']

    def test_get_produit_inexistant_retourne_404(self, client):
        response = client.get('/products/99999')
        assert response.status_code == 404

    def test_schema_complet(self, client, api_product):
        """La réponse contient tous les champs attendus."""
        response = client.get(f'/products/{api_product["id"]}')
        data = response.json()
        for field in ['id', 'name', 'price', 'stock', 'active', 'created_at']:
            assert field in data, f'Champ manquant : {field}'
```

## 2.2 Tests de création, mise à jour et suppression

```
tests/integration/test_products_api.py — écriture
@pytest.mark.integration
class TestCreateProduct:

    def test_creation_valide(self, client):
        payload = {'name': 'Souris Ergonomique', 'price': 49.99, 'stock': 30}
        response = client.post('/products/', json=payload)
        assert response.status_code == 201
        data = response.json()
        assert data['name'] == 'Souris Ergonomique'
        assert data['price'] == 49.99
        assert data['id'] is not None

    def test_creation_prix_negatif_422(self, client):
        response = client.post('/products/', json={'name': 'X', 'price': -10.0, 'stock': 5})
        assert response.status_code == 422

    def test_creation_nom_vide_422(self, client):
        response = client.post('/products/', json={'name': '', 'price': 10.0, 'stock': 5})
        assert response.status_code == 422

    def test_creation_stock_negatif_422(self, client):
        response = client.post('/products/', json={'name': 'T', 'price': 10.0, 'stock': -1})
        assert response.status_code == 422

    def test_active_true_par_defaut(self, client):
        response = client.post('/products/', json={'name': 'Actif', 'price': 1.0, 'stock': 1})
        assert response.json()['active'] is True


@pytest.mark.integration
class TestUpdateDeleteProduct:

    def test_mise_a_jour_prix(self, client, api_product):
        pid = api_product['id']
        response = client.put(f'/products/{pid}', json={'price': 79.99})
        assert response.status_code == 200
        assert response.json()['price'] == 79.99

    def test_mise_a_jour_stock(self, client, api_product):
        pid = api_product['id']
        response = client.put(f'/products/{pid}', json={'stock': 100})
        assert response.status_code == 200
        assert response.json()['stock'] == 100

    def test_suppression_soft_delete(self, client):
        """DELETE désactive le produit (soft delete) — il n'est plus accessible."""
        create = client.post('/products/', json={'name': 'A supprimer', 'price': 1.0, 'stock': 1})
        pid = create.json()['id']
        response = client.delete(f'/products/{pid}')
        assert response.status_code == 204
        # Vérifier que le produit n'est plus accessible
        get_resp = client.get(f'/products/{pid}')
        assert get_resp.status_code == 404

#test_filtre_prix_min_max() reponse Q2.4 (à faire)
def test_filtre_prix_min_max(self, client):
    """GET /products/?min_price=50&max_price=200 retourne seulement les produits
    dont le prix est entre 50€ et 200€ inclus."""
    # Créer 3 produits avec des prix différents
    # Filtrer : min=50, max=200
    # Vérifier que tous les résultats sont dans la fourchette
    # Vérifier que 'Dans la fourchette' (100€) est bien présent
    # Vérifier que les produits hors fourchette sont absents
```

## Q2 - Tests /products

2.1.Implémentez tous les tests de TestListProducts, TestGetProduct, TestCreateProduct et TestUpdateDeleteProduct. Lancez pytest tests/integration/test_products_api.py -v et commentez la sortie.

2.2.Ajoutez un test test_filtre_prix_min_max() qui vérifie que GET /products/?min_price=50&max_price=200 retourne uniquement des produits dont le prix est compris entre 50 et 200€.

# Partie 3 : Tests du Scénario Commande Complet

Les tests d'intégration les plus importants de ShopFlow : ils valident le flux complet **produit → panier → commande**, y compris la décrémentation du stock et le vidage automatique du panier.

## 3.1 Tests du panier /cart

```
tests/integration/test_cart_api.py
# tests/integration/test_cart_api.py
import pytest

@pytest.mark.integration
class TestCart:

    def _creer_produit(self, client, stock=10):
        """Helper : crée un produit de test."""
        r = client.post('/products/', json={'name': 'Produit Test', 'price': 50.0, 'stock': stock})
        assert r.status_code == 201
        return r.json()

    def test_ajout_produit_au_panier(self, client):
        p = self._creer_produit(client)
        response = client.post(f'/cart/?user_id=100', json={
            'product_id': p['id'], 'quantity': 2
        })
        assert response.status_code == 201
        data = response.json()
        assert data['user_id'] == 100
        assert len(data['items']) == 1
        assert data['items'][0]['quantity'] == 2

    def test_sous_total_calcule(self, client):
        """50€ HT × 2 × 1.20 TVA = 120€ TTC."""
        p = self._creer_produit(client)
        client.post(f'/cart/?user_id=101', json={'product_id': p['id'], 'quantity': 2})
        response = client.get('/cart/101')
        assert response.status_code == 200
        assert response.json()['sous_total'] == pytest.approx(120.0, rel=1e-2)

    def test_ajout_stock_insuffisant_400(self, client):
        p = self._creer_produit(client, stock=0)   # produit en rupture
        response = client.post(f'/cart/?user_id=102', json={
            'product_id': p['id'], 'quantity': 1
        })
        assert response.status_code == 400

    def test_ajout_meme_produit_incremente_quantite(self, client):
        p = self._creer_produit(client, stock=20)
        client.post(f'/cart/?user_id=103', json={'product_id': p['id'], 'quantity': 2})
        client.post(f'/cart/?user_id=103', json={'product_id': p['id'], 'quantity': 3})
        cart = client.get('/cart/103').json()
        assert cart['items'][0]['quantity'] == 5   # 2 + 3

    def test_vider_panier(self, client):
        p = self._creer_produit(client)
        client.post(f'/cart/?user_id=104', json={'product_id': p['id'], 'quantity': 1})
        response = client.delete('/cart/104')
        assert response.status_code == 204
        cart = client.get('/cart/104').json()
        assert cart['items'] == []
```

## 3.2 Tests des commandes /orders scénario complet

```
tests/integration/test_orders_api.py
# tests/integration/test_orders_api.py
```

```python
import pytest

@pytest.mark.integration
class TestOrders:

    def _setup_panier(self, client, user_id, stock=10, price=100.0, quantity=2):
        """Helper : crée produit + ajoute au panier."""
        p = client.post('/products/', json={
            'name': 'Produit Commande', 'price': price, 'stock': stock
        }).json()
        client.post(f'/cart/?user_id={user_id}', json={
            'product_id': p['id'], 'quantity': quantity
        })
        return p

    def test_creation_commande_valide(self, client):
        self._setup_panier(client, user_id=200)
        response = client.post('/orders/', json={'user_id': 200})
        assert response.status_code == 201
        data = response.json()
        assert data['status'] == 'pending'
        assert data['total_ttc'] > 0

    def test_total_ttc_correct(self, client):
        """100€ HT × 2 × 1.20 = 240€ TTC."""
        self._setup_panier(client, user_id=201, price=100.0, quantity=2)
        order = client.post('/orders/', json={'user_id': 201}).json()
        assert order['total_ht'] == pytest.approx(200.0, rel=1e-2)
        assert order['total_ttc'] == pytest.approx(240.0, rel=1e-2)

    def test_commande_decremente_stock(self, client):
        """Après commande de 2, le stock passe de 10 à 8."""
        p = self._setup_panier(client, user_id=202, stock=10, quantity=2)
        client.post('/orders/', json={'user_id': 202})
        updated = client.get(f'/products/{p["id"]}').json()
        assert updated['stock'] == 8

    def test_commande_vide_le_panier(self, client):
        self._setup_panier(client, user_id=203)
        client.post('/orders/', json={'user_id': 203})
        cart = client.get('/cart/203').json()
        assert cart['items'] == []

    def test_panier_vide_retourne_400(self, client):
        response = client.post('/orders/', json={'user_id': 9999})
        assert response.status_code == 400

    def test_commande_avec_coupon(self, client, api_coupon):
        """100€ HT × 1 × 1.20 = 120€ TTC → -10% = 108€."""
        p = client.post('/products/', json={'name': 'PC', 'price': 100.0, 'stock': 5}).json()
        client.post(f'/cart/?user_id=204', json={'product_id': p['id'], 'quantity': 1})
        order = client.post('/orders/', json={
            'user_id': 204, 'coupon_code': api_coupon['code']
        }).json()
        assert order['total_ttc'] == pytest.approx(108.0, rel=1e-2)
        assert order['coupon_code'] == api_coupon['code']

    def test_statut_pending_vers_confirmed(self, client):
        self._setup_panier(client, user_id=205)
        order = client.post('/orders/', json={'user_id': 205}).json()
        response = client.patch(f'/orders/{order["id"]}/status',
                                json={'status': 'confirmed'})
        assert response.status_code == 200
        assert response.json()['status'] == 'confirmed'

    def test_transition_invalide_400(self, client):
        """pending → shipped directement est interdit."""
        self._setup_panier(client, user_id=206)
        order = client.post('/orders/', json={'user_id': 206}).json()
        response = client.patch(f'/orders/{order["id"]}/status',
                                json={'status': 'shipped'})
        assert response.status_code == 400
```

```
#Q3.3 test_coupon_inexistant_retourne_404()- a faire dans test_orders_api.py   (à faire)
def test_coupon_inexistant_retourne_404(self, client):
    """Un coupon inexistant lors de la commande → 404."""
    # 1) créer un panier avec un produit
    # 2) tenter de créer une commande avec un coupon_code inexistant
    # 3) vérifier que la réponse est 404 et que le message d'erreur mentionne le code du coupon inexistant

#Q3.4 test_get_commande_par_id()- a faire dans test_orders_api.py (à faire)
def test_get_commande_par_id(self, client):
    """Créer une commande puis la récupérer par son id."""
    # 1) créer une commande
    #2) récupérer la commande par son id
    #3) vérifier que les données sont correctes (id, total_ttc, status)
```

**Q3 : Tests Panier & Commandes**

3.1. Implémentez la suite TestCart complète. Lancez pytest tests/integration/test_cart_api.py -v et copiez et commentez la sortie.

3.2. Implémentez la suite TestOrders complète. Vérifiez que test_total_ttc_correct() passe avec total_ttc == pytest.approx(240.0). Copiez la sortie pytest.

3.3. Ajoutez le test test_coupon_inexistant_retourne_404() qui vérifie que POST /orders/ avec un coupon_code='FAKECODE' inexistant retourne bien 404.

3.4. Ajoutez le test test_get_commande_par_id() qui crée une commande, récupère son id, appelle GET /orders/{id} et vérifie que le même id et le même total_ttc sont retournés.

# Partie 4 : Données de Test & Faker

Les tests robustes ne reposent pas sur des données codées en dur. **faker** génère des données réalistes et variées à chaque exécution, ce qui augmente la couverture des cas limites.

## 4.1 Utiliser faker dans les fixtures

tests/conftest.py ( fixtures faker)

```
# Ajouter dans tests/conftest.py
from faker import Faker

fake = Faker('fr_FR')   # données en français

@pytest.fixture
def fake_product_data():
    """Génère un payload produit réaliste et aléatoire."""
    return {
        'name':     fake.catch_phrase()[:50],   # ex: 'Synergistic Rubber Chair'
        'price':    round(fake.pyfloat(min_value=1, max_value=2000, right_digits=2), 2),
        'stock':    fake.random_int(min=0, max=500),
        'category': fake.random_element(['informatique', 'peripheriques', 'audio', 'gaming']),
        'description': fake.sentence(nb_words=10),
    }

@pytest.fixture
def multiple_products(client):
    """Crée 5 produits avec faker pour tester la liste et les filtres."""
    faker_inst = Faker()
    products = []
    for i in range(5):
        r = client.post('/products/', json={
            'name': faker_inst.word().capitalize() + f' {i}',
            'price': round(10.0 + i * 20, 2),
            'stock': 10,
        })
        products.append(r.json())
    yield products
    # Cleanup : désactiver les 5 produits
    for p in products:
        client.delete(f'/products/{p["id"]}')
```

## 4.2 Tests utilisant faker

tests/integration/test_faker_products.py

```
# tests/integration/test_faker_products.py
import pytest

@pytest.mark.integration
class TestProductsAvecFaker:

    def test_creation_donnees_faker(self, client, fake_product_data):
        """Crée un produit avec des données faker et vérifie la réponse."""
        response = client.post('/products/', json=fake_product_data)
        assert response.status_code == 201
        data = response.json()
        assert data['name'] == fake_product_data['name']
        assert data['price'] == fake_product_data['price']

    def test_liste_avec_multiple_produits(self, client, multiple_products):
        """Vérifie que les 5 produits créés apparaissent dans la liste."""
        response = client.get('/products/')
        assert response.status_code == 200
        ids_liste = [p['id'] for p in response.json()]
        for p in multiple_products:
            assert p['id'] in ids_liste

    @pytest.mark.parametrize('prix,attendu_422', [
```

```
        (0.0, True),     # prix = 0 → invalide (gt=0)
        (-1.0, True),    # prix négatif → invalide
        (0.01, False),   # prix minimal valide
        (9999.99, False), # prix élevé mais valide
    ])
    def test_validation_prix(self, client, prix, attendu_422):
        response = client.post('/products/', json={'name': 'Test', 'price': prix, 'stock': 1})
        if attendu_422:
            assert response.status_code == 422
        else:
            assert response.status_code == 201

    # Q4.3 test_noms_longs() - (à faire)

    def test_noms_longs(self, client):

        """Un nom de 101 chars dépasse max_length=100 → 422."""

        # 1)créer un produit avec un nom de 101 caractères → doit retourner 422 (erreur de validation)

        # 2) vérifier que le message d'erreur mentionne le champ 'name' et la contrainte de longueur

        # 3)créer un produit avec un nom de 100 caractères → doit réussir (201) car c'est la limite max
```

**Q4 - Données de Test & Faker**

**4.1.** Ajoutez les fixtures fake_product_data et multiple_products dans conftest.py. Implémentez les tests de TestProductsAvecFaker et lancez-les. Copiez la sortie.

**4.2.** Ajoutez un test test_noms_longs() qui vérifie que POST /products/ avec name='A'*101 (101 caractères) retourne bien 422 (la contrainte max_length=100 est respectée).

**4.3.** Expliquez ce que fait Faker('fr_FR') par rapport à Faker() par défaut. Dans quel contexte les données localisées sont-elles importantes pour les tests de ShopFlow ?

# Partie 5 : Rapport JUnit XML & Smoke Tests

## 5.1 Générer un rapport JUnit XML

Jenkins consomme des rapports JUnit XML pour afficher les résultats de tests dans son interface. Générez ce rapport avec l'option `--junitxml` :

```
Terminal
# Lancer les tests d'intégration avec rapport JUnit XML
pytest tests/integration/ -v \
    --junitxml=junit-integration.xml \
    --no-cov                          # pas de coverage ici

# Lancer tous les tests avec coverage + JUnit
pytest tests/ \
    --junitxml=junit-report.xml \
    --cov=app \
    --cov-report=xml:coverage.xml \
    --cov-report=html \
    --cov-fail-under=80

# Vérifier le contenu du rapport XML
cat junit-integration.xml | head -30
```

**Structure d'un rapport JUnit XML**

- <testsuites> : racine du document, contient les statistiques globales (tests, failures, errors, time)
- <testsuite name='tests/integration/test_products_api.py'> : un bloc par fichier de test
- <testcase classname='TestListProducts' name='test_liste_vide_au_demarrage'> : un bloc par test
- <failure message='...'> : présent uniquement si le test a échoué - Jenkins affiche ce message
- Jenkins : stage junit 'junit-report.xml' pour publier le rapport dans l'interface

## 5.2 Smoke tests : /health

Les smoke tests vérifient que l'API est opérationnelle après chaque déploiement. Créez le fichier `tests/integration/test_health.py` :

```
tests/integration/test_health.py
# tests/integration/test_health.py
import pytest

@pytest.mark.integration
class TestHealth:

    def test_health_ok(self, client):
        """Smoke test — l'API doit répondre status=ok."""
        response = client.get('/health')
        assert response.status_code == 200
        data = response.json()
        assert data['status'] == 'ok'
        assert data['db'] == 'sqlite'
        assert 'version' in data

    def test_root_accessible(self, client):
        response = client.get('/')
        assert response.status_code == 200

    def test_docs_swagger_accessible(self, client):
        response = client.get('/docs')
        assert response.status_code == 200

    def test_openapi_json_accessible(self, client):
        """Le schéma OpenAPI doit être disponible."""
        response = client.get('/openapi.json')
        assert response.status_code == 200
```

```
        data = response.json()
        assert data['info']['title'] == 'ShopFlow API'
```

## 5.3 Lancer la suite complète

```
Terminal — suite complète
# Suite complète intégration avec rapport XML
pytest tests/integration/ -v --junitxml=junit-integration.xml

# Résultat attendu :
# tests/integration/test_health.py::TestHealth::test_health_ok PASSED
# tests/integration/test_products_api.py::TestListProducts::test_liste_vide PASSED
# tests/integration/test_orders_api.py::TestOrders::test_creation_commande PASSED
# ...
# NN passed in X.XXs
```

**Q5 - JUnit XML & Synthèse**

5.1.Implémentez les smoke tests de TestHealth. Lancez pytest tests/integration/ -v --junitxml=junit.xml. Copiez la sortie terminal et les 15 premières lignes du fichier junit.xml généré.

5.2. Lancez pytest tests/ --cov=app --cov-report=term-missing --no-header -q. Votre coverage total doit être ≥ 80%. Copiez la sortie. Si < 80%, identifiez les fichiers à couvrir et ajoutez les tests manquants.

5.3. Expliquez la différence entre un test d'intégration et un smoke test. Dans quel ordre Jenkins doit-il les exécuter et pourquoi ?